

Pequeno roteiro para entender o algoritmo de Adam

Referência do artigo original: Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." arXiv preprint arXiv:1412.6980 (2014).

Para facilitar a intuição acerca do algoritmo de otimização de Adam, pequenos segmentos de código em linguagem Octave são inseridos ao longo do texto, e podem ser facilmente executados. Por exemplo:

Suponha que um sinal x_0 é observado a instantes de tempo discretos, $t = 1, 2, 3, \dots$, e que esse sinal deveria ser constante, mas está impregnado com um ruído aditivo de medição. Isso pode ser simulado, para 100 instantes de observação, e para um ruído aleatório aditivo, normal e branco da seguinte forma, no Octave:

```
x0 = 4.5; # O valor verdadeiro de g pode ser 4.5 Volts, por exemplo
x = x0 + randn(1,100); # 100 amostras ruidosas de x ao longo de 100 instantes de tempo
plot(x)
```

Como o ruído aditivo é de média nula, então podemos estimar x_0 calculando a média das 100 amostras:

```
x0_estim = sum(x)/100
```

No caso deste pequeno problema, x_0_estim é a melhor estimativa que podemos ter de x_0 . No entanto, se as amostras são obtidas sequencialmente, e desejamos estimar x_0 sem espera, após a chegada da n -ésima amostra, então podemos fazer a seguinte manipulação algébrica:

$$x_0_estim(n) = (x(1)+x(2)+\dots+x(n))/n = x(n)/n + (x(1)+\dots+x(n-1))/n$$

mas como $x_0_estim(n-1) = (x(1)+\dots+x(n-1))/(n-1)$, então podemos substituir $(x(1)+\dots+x(n-1))/n$ por $((n-1)/n) x_0_estim(n-1)$ na equação acima:

$$x_0_estim(n) = x(n)/n + ((n-1)/n) x_0_estim(n-1)$$

ou ainda

$$x_0_estim(n) = (1/n) x(n) + ((n-1)/n) x_0_estim(n-1)$$

No Octave, podemos experimentar esse estimador sequencial, assim:

```
plot(x)
hold on
x0_estim_anterior = 0; # Ainda não há nenhuma observação
for n = 1:100
    x0_estim(n) = (1/n)*x(n) + ((n-1)/n)*x0_estim_anterior;
    x0_estim_anterior = x0_estim(n);
end
plot(x0_estim)
```

Observe como rapidamente $x0_estim$ se aproxima da melhor estimaco possvel para essas 100 amostras do sinal, que  $\text{sum}(x)/100$. Observe tambm que $\text{sum}(x)/100$  exatamente o ltimo valor estimado, para $n = 100$.

Esse estimador pode ser dito no-viesado, mas no algoritmo de Adam, uma verso viesada dele  usada, em que $((n-1)/n)$  substituído por uma constante real β , entre 0 e 1, e $1/n$  substituído por $(1-\beta)$ (cabe notar que, assim como β e $(1-\beta)$, os pesos $(1/n)$ e $(n-1)/n$ tambm somam 1). Assim, temos uma alternativa de estimaco viesada

$$x0_estim(n) = (1-\beta) x(n) + \beta x0_estim(n-1)$$

por exemplo, no Octave, se escolhermos $\beta = 0.9$, temos:

```
beta = 0.9;
x0_estim_anterior = 0; # Ainda no h nenhuma observaco
for n = 1:100
    x0_estim(n) = (1-beta)*x(n) + beta*x0_estim_anterior;
    x0_estim_anterior = x0_estim(n);
end
plot(x0_estim)
```

Observe como esse estimador demora a fornecer estimativas razoveis de $x0$, pois ele  enviesado para valores prximos a 0 nos primeiros instantes de amostragem. Isso  provocado pela imposio de um valor constante β em lugar do peso dinmico $((n-1)/n)$. Para compensar isso, mantendo esse parmetro constante, o algoritmo de Adam inclui uma etapa de compensaco do vis, da seguinte maneira:

Se o sinal $x(n)$ fosse "limpo", isto , se o rudo de medio fosse nulo, ento a primeira amostra, $x(1)$, seria igual a $x0$, e usando o estimador a parmetro constante β , essa observaco sozinha contribuiria para uma estimativa de $x0$ no tempo n igual a $x0*(1-\beta)\beta^{(n-1)}$ (verifique isso usando a rotina em Octave acima, se achar necessrio). Da mesma forma, a segunda amostra, $x(2)$, seria tambm igual a $x0$, e, sozinha, contribuiria para uma estimativa de $x0$ no tempo n igual a $x0*(1-\beta)\beta^{(n-2)}$. Seguindo essa linha de raciocnio, conclumos que, sem rudo, no instante n , todas as entradas constantes iguais a $x0$, geram uma estimativa de $x0$ que  a seguinte soma:

$$x0_estim(n) = \text{sum}_{\{i=0^{n-1}\}}(x0 (1-\beta)\beta^i) = x0 (1-\beta) \text{sum}_{\{i=0^{n-1}\}}(\beta^i)$$

onde a soma de progresso geomtrica pode ser simplificada:

$$x0_estim(n) = x0 (1-\beta) (1+\beta+\dots+\beta^{(n-1)}) = x0 (1-\beta) (1-\beta^n)/(1-\beta)$$

$$x0_estim(n) = x0 (1-\beta^n)$$

Ou seja, mesmo sem rudo, no instante n , o valor estimado de $x0$  estimado como $(1-\beta^n) x0$, onde o fator $(1-\beta^n)$ pode ser visto como um vis. Assim, nos termos usados no artigo em que o algoritmo Adam  proposto, podemos corrigir esse vis apenas dividindo o valor estimado no instante n , $x0_estim(n)$, por $(1-\beta^n)$:

$$x0_estim_semVies(n) = x0_estim(n) / (1-\beta^n)$$

```
beta = 0.9;
x0_estim_anterior = 0; # Ainda não há nenhuma observação
for n = 1:100
    x0_estim(n) = (1-beta)*x(n) + beta*x0_estim_anterior;
    x0_estim_anterior = x0_estim(n);
    x0_estim_semVies(n) = x0_estim(n)/(1-beta^n);
end
plot(x0_estim_semVies)
```

Observe como esse novo estimador rapidamente se aproxima do valor real de x_0 . Para finalizar, observe que o Algoritmo 1, no artigo em que o Adam é proposto, usa esse estimador para estimar a média do gradiente no instante de amostragem t , denotada como m_t (o chapéu em m representa “bias-corrected estimate”) e a média do quadrado do gradiente denotada como v_t . Assim, o ajuste de pesos/parâmetros, no gradiente estocástico, pode substituir o mero gradiente no instante t pela média desse gradiente, normalizada pela raiz quadrada de v_t .